



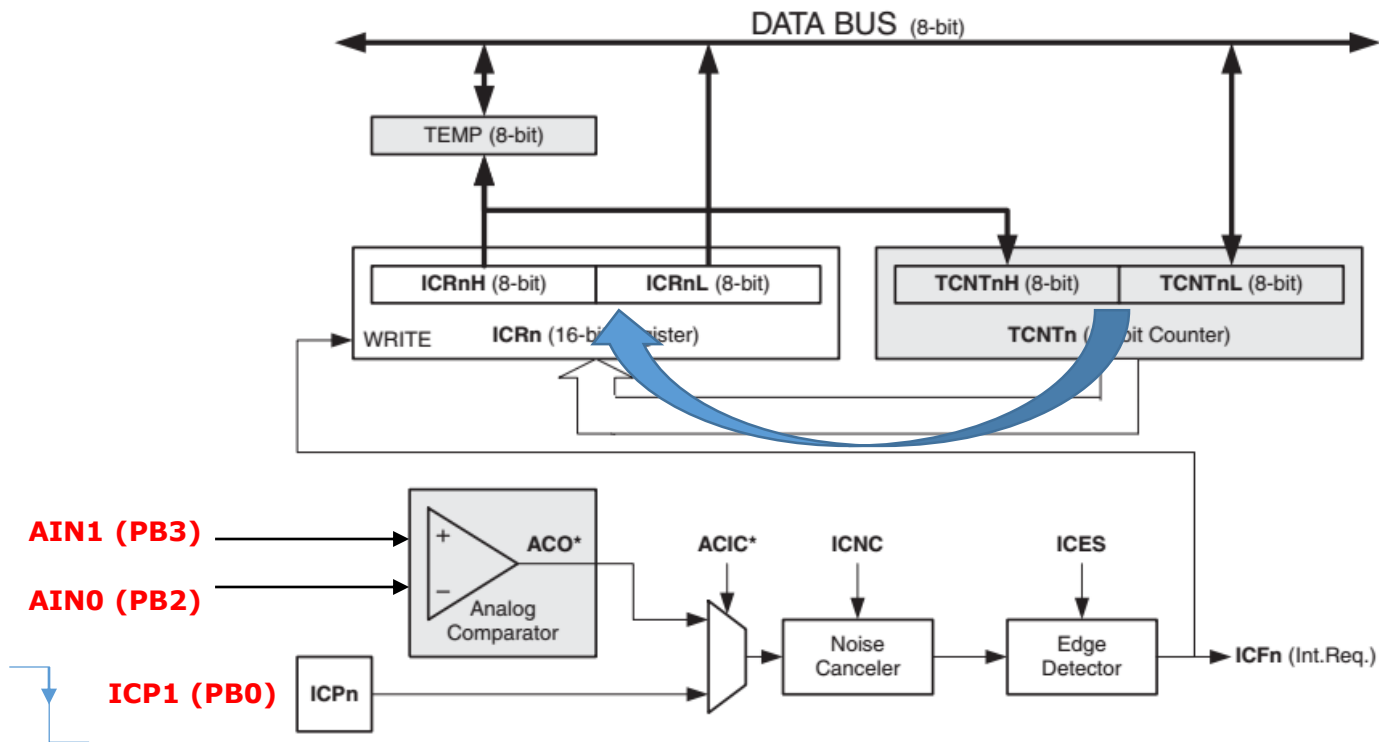
CIRCUITOS DIGITALES Y MICROCONTROLADORES 2026

Facultad de Ingeniería
UNLP

Periférico TIMER1 (continuación)

Ing. José Juárez

TIMER 1: Captura de entrada



- Cuando ocurre un flanco en el terminal ICP1 (PB0) se dispara una captura que copia el TCNT1 en el ICR1, se activa el flag ICF1 y si esta habilitada (TICIE1=1) se genera una interrupción de aviso de captura
- El comparador analógico, compara la tensión de AIN1(PB3) contra AIN0 (PB2) y si es mayor da una salida "1"

TIMER 1: Captura de entrada

- Registros de configuración y control:

7	6	5	4	3	2	1	0	
COM1A1	COM1A0	COM1B1	COM1B0	—	—	WGM11	WGM10	TCCR1A
R/W	R/W	R/W	R/W	R	R	R/W	R/W	
0	0	0	0	0	0	0	0	

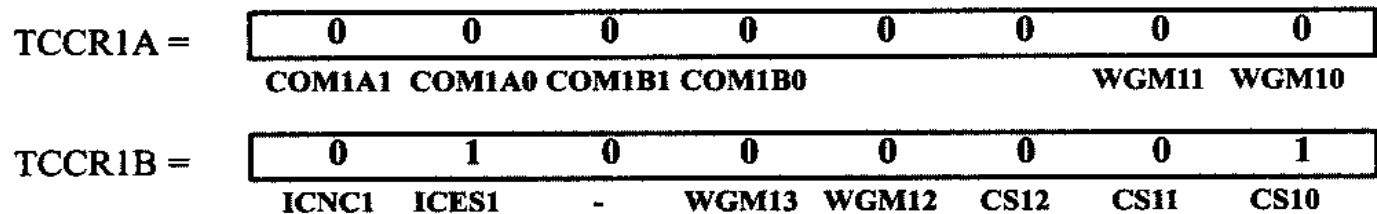
7	6	5	4	3	2	1	0	
ICNC1	ICES1	—	WGM13	WGM12	CS12	CS11	CS10	TCCR1B
R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
0	0	0	0	0	0	0	0	

- ICNC1**: Noise canceler enable (4 samples @fcpu deben ser iguales para confirmar el flanco)
- ICES1**: selección de flanco 0: bajada, 1: subida
- WGM1x**: ver modos de funcionamientos en tabla de Modos.
- CS1x**: clock select

CS12	CS11	CS10	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	clk _{I/O} /1 (No prescaling)
0	1	0	clk _{I/O} /8 (From prescaler)
0	1	1	clk _{I/O} /64 (From prescaler)
1	0	0	clk _{I/O} /256 (From prescaler)
1	0	1	clk _{I/O} /1024 (From prescaler)
1	1	0	External clock source on T1 pin. Clock on falling edge.
1	1	1	External clock source on T1 pin. Clock on rising edge.

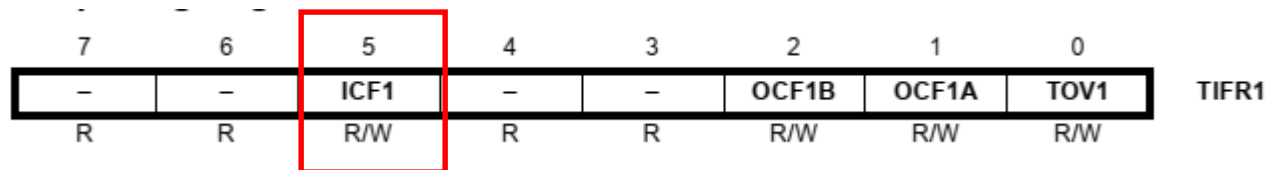
TIMER 1: Captura de entrada

- Configuración del Capturador:



Modo Normal, sin prescaler, sin cancelador de ruido, con captura en flanco +

Registro de Flags TIFR1:



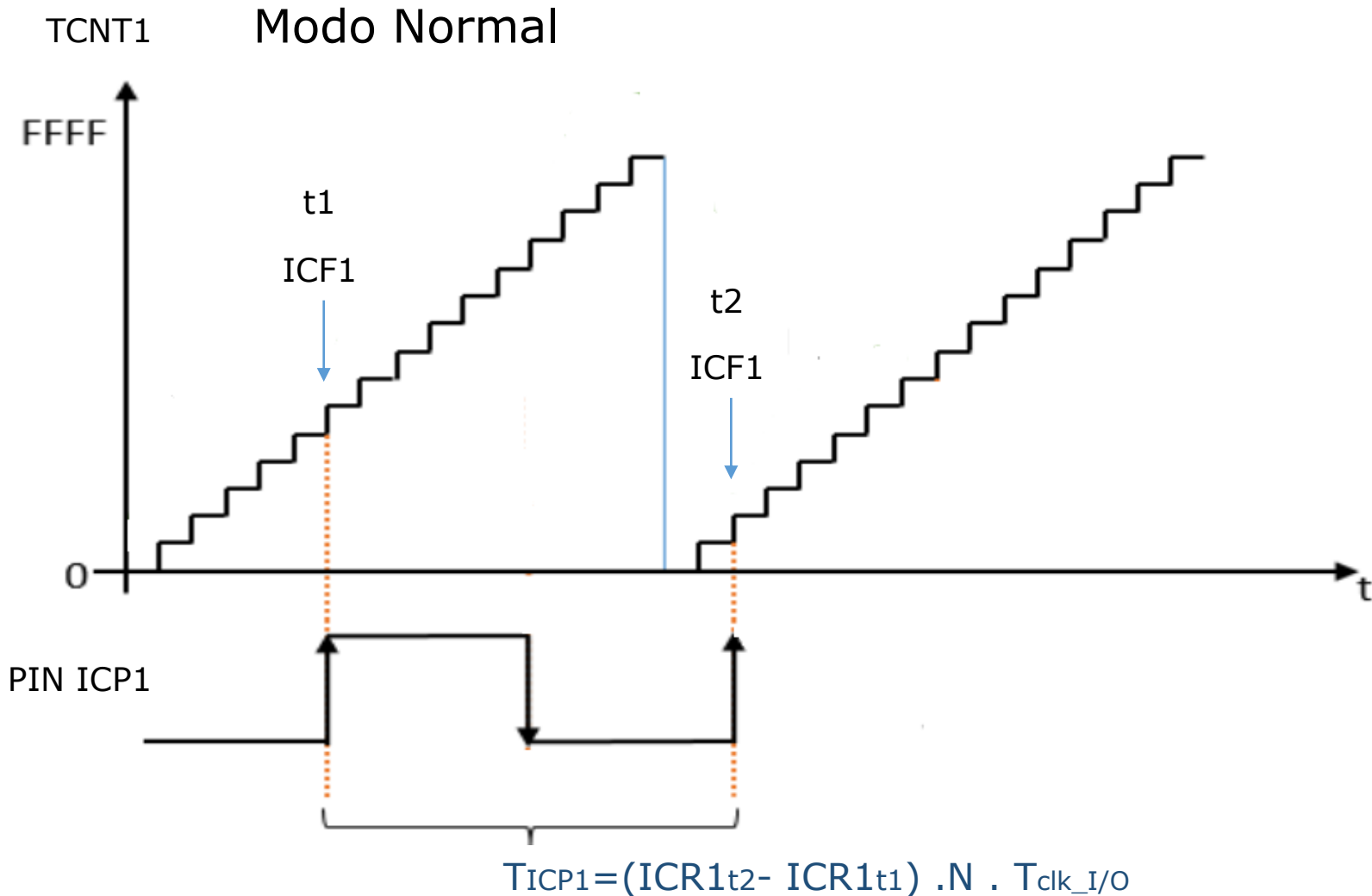
El Flag ICF1 se activará cuando ocurra un flanc + en terminal ICP1

Luego se debe borrar escribiendo un "1": $TIFR1 \mid= (1 \ll ICF1)$;

O se borra automáticamente cuando se ejecuta el handler de interrupción

TIMER 1: Captura de entrada

Ejemplo: medición de periodo de una señal digital incógnita



TIMER 1: IC

$$TICP1 = (ICR1_{t2} - ICR1_{t1}) \cdot N \cdot T_{clk_I/O}$$

- Ejemplo: **Medición de período** y mostrar en PORTC y D

```
#include <avr/io.h>
int main(void)
{
    uint16_t t, periodo;
    DDRB &= ~(1<<PB0);           //ICP1 =entrada
    DDRC = 0xFF;                  //PORTC =salida
    DDRD = 0xFF;                  //PORTD =salida

    TCCR1A = 0;                   //Modo normal
    TCCR1B = (1<<ICES1) | (1<<CS10); //ICP1 flaco + No prescaler

    while(TIFR1 & (1<<ICF1) == 0);
    t = ICR1;
    TIFR1 = (1<<ICF1);
    while(TIFR1 & (1<<ICF1) == 0);
    periodo = ICR1 - t;
    TIFR1 = (1<<ICF1);

    PORTD = periodo;              // Low byte
    PORTC = periodo >> 8;        //High byte

    while(1);
return 0;
}
```

TIMER 1: IC

$$T_{ICP1} = (ICR1_{t2} - ICR1_{t1}) \cdot N \cdot T_{clk_I/O}$$

- ¿Cuántos valores distintos puedo medir?

Para $f_{clk_I/O}$ y N fijos $\Rightarrow$ $ICR1$ variable (16bits)
por lo tanto 65535 valores diferentes de T_{ICP1} o f_{ICP1}

- Cuál es el mínimo periodo?

$$\Rightarrow |ICR1_{t2} - ICR1_{t1}| = 1 \Rightarrow T_{ICP1_Min} = 1 \cdot N \cdot T_{clk_I/O}$$

- Cuál es el máximo periodo?

$$\Rightarrow |ICR1_{t2} - ICR1_{t1}| = 65535 \Rightarrow T_{ICP1_Max} = 65535 \cdot N \cdot T_{clk_I/O}$$

- Cuál es la resolución del método?

Para $f_{clk_I/O}$ y N fijos $\Rightarrow$ diferencia entre dos periodos medidos

$$T_{ICP1_x} = DIF_x \cdot N \cdot T_{clk_I/O}$$

$$T_{ICP1_y} = DIF_y \cdot N \cdot T_{clk_I/O}$$

$$\Rightarrow \Delta T = T_{ICP1_y} - T_{ICP1_x} = N \cdot T_{clk_I/O} \cdot (DIF_y - DIF_x)$$

La mínima diferencia es $\cdot (DIF_y - DIF_x) = \pm 1$

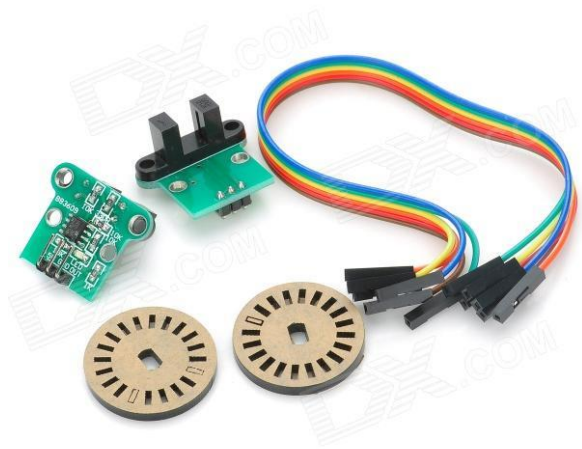
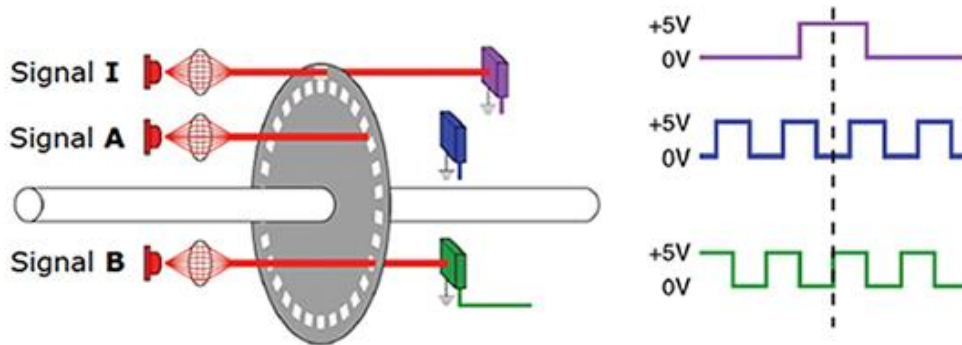
$$\Rightarrow \text{La resolución es } \Delta T_{min} = \pm N \cdot T_{clk_I/O}$$

$$\text{Y la resolución relativa en \% es } \Delta T_{min} / T_{ICP1} = \pm 100 / |ICR1_{t2} - ICR1_{t1}|$$

TIMER 1: IC

$$T_{ICP1} = (ICR1_{t2} - ICR1_{t1}) \cdot N \cdot T_{clk_I/O}$$

- Ejemplo: Medición de RPM de un motor con un encoder



A y B desfasados 1/4 de vuelta para detectar el sentido de giro
Supongamos que el encoder tiene 25 pulsos/vuelta, entonces

$$1 \text{ vuelta tarda } 25 \cdot T_{pulso} [\text{seg}]$$

por lo tanto en 1 minuto tendremos:

$$RPM = 60 / (25 \cdot T_{pulso})$$

Si utilizamos la entrada de captura para medir entonces $T_{pulso} = T_{ICP1}$

Por lo tanto:

$$RPM = 60 / (25 \cdot DIF_{ICR1} \cdot N \cdot T_{clk_I/O})$$

TIMER 1: IC

$$RPM = 60 / (25 (ICR1t2 - ICR1t1) N T_{clk_I/O})$$

- calcular el rango y la resolución del valor en RPM

$$RPM[rpm] = \frac{60 f_{clk_{IO}}}{25 DIF N}$$

$$RPM_{min}[rpm] = \frac{60 f_{clk_{IO}}}{25 DIF N} = \frac{60 f_{clk_{IO}}}{25.65535 N}$$

$$RPM_{max}[rpm] = \frac{60 f_{clk_{IO}}}{25 DIF N} = \frac{60 f_{clk_{IO}}}{25 N}$$

$$DIF' = DIF + 1LSB$$

$$\Delta RPM_{min}[rpm] = \frac{60 f_{clk_{IO}}}{25 DIF' N} - \frac{60 f_{clk_{IO}}}{25 DIF N} \cong \pm \frac{60 f_{clk_{IO}}}{25 DIF^2 N}$$

$$\frac{\Delta RPM_{min}}{RPM} [\%] \cong \pm \frac{100}{DIF}$$

Lo que significa que debo seleccionar el prescaldor N para diferencias grandes . Por ejemplo si quiero medir con resolución $< \pm 0,1\%$ entonces $|ICR1t2 - ICR1t1| > 1000$

TIMER 1: IC

$$RPM = 60 / (25 (dif) N T_{clk_I/O})$$

- calcular la resolución si queremos medir 180 RPM
- Si fclk=16MHz, veamos como influye N

$$dif = \frac{60 \cdot 16000000}{25 \cdot 180 \cdot N}$$

$$\Delta RPM_{min}[rpm] \cong \pm \frac{60 \cdot 16000000}{25 \cdot dif^2 \cdot N}$$

PRE	dif	ΔRPMmin	Algoritmo Mide dif=	Algoritmo calcula rpm=	LCD muestra
1	213333,3333	0,00084	-	-	-
8	26666,6666	0,00675	26666 – 26667	180,0045 – 179,9977	180,00 - 179,99
64	3333,3333	0,054	3333 -3334	180,018 - 179,964	180,01 - 179,96
256	833,3333	0,216	833 – 834	180,072 - 179,856	180,07 - 179,85
1024	208,3333	0,864	208 -209	180,288 – 179,425	180,28 - 179,42

TIMER 1: IC

Ejemplo: **Medición de ancho de pulso** (asumiendo que el valor a medir cabe en 8 bits)

```
#include <avr/io.h>
int main(void)
{
    uint16_t t, apulso;
    DDRB &= ~(1<<PB0);           //ICP1 =entrada
    DDRC = 0xFF;                  //PORTC =salida
    DDRD = 0xFF;                  //PORTD =salida

    TCCR1A = 0;                   //Modo normal
    TCCR1B = (1<<ICES1) | (1<<CS10); //ICP1 flaco + No prescaler

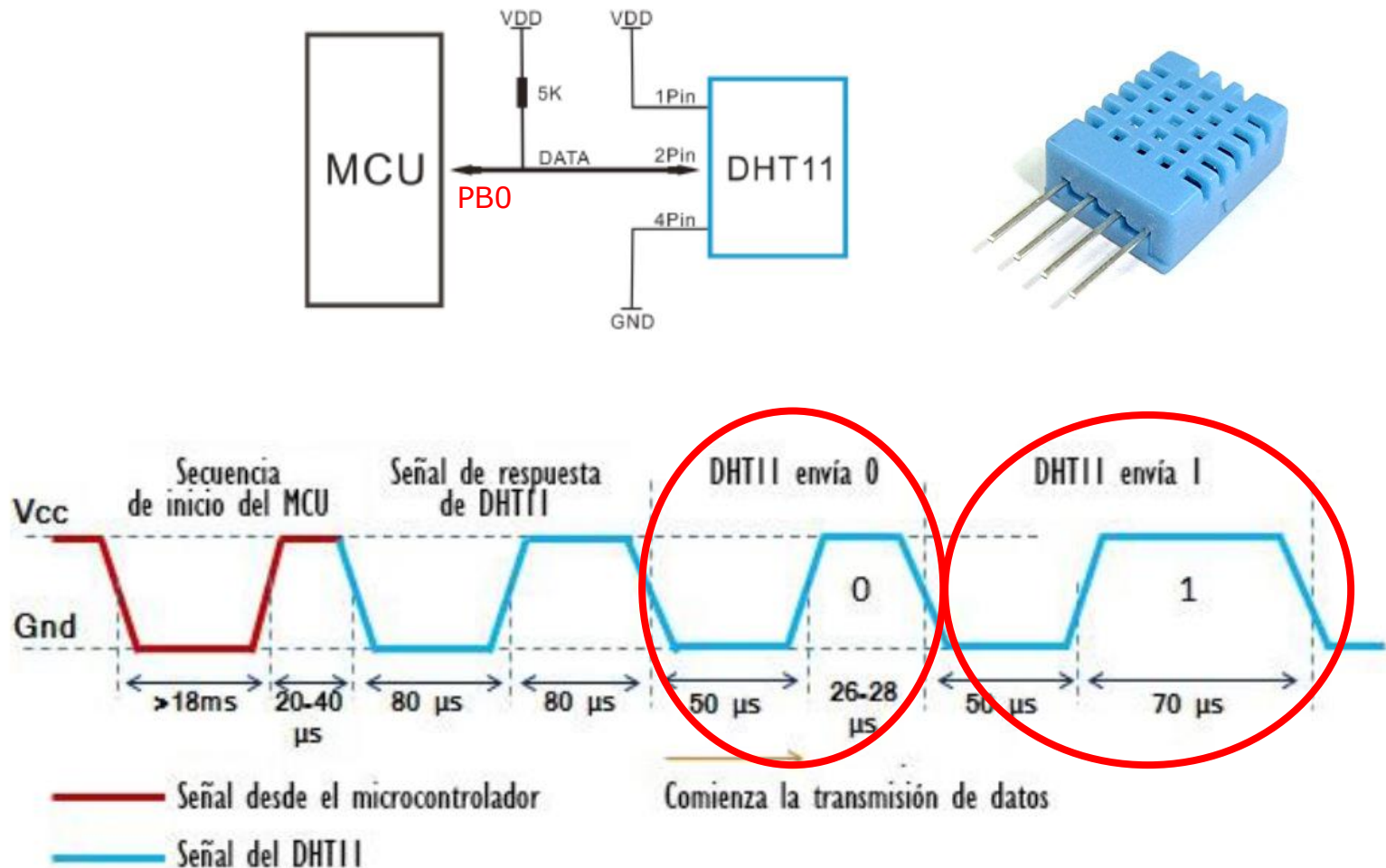
    while (TIFR1 & (1<<ICF1) == 0);
    t = ICR1;
    TIFR1 = (1<<ICF1);
    TCCR1B &= ~(1<<ICES1); //ICP1 flaco -
    while (TIFR1 & (1<<ICF1) == 0);
    apulso = ICR1 - t;
    TIFR1 = (1<<ICF1);

    PORTD = apulso;               // Low byte
    PORTC = apulso >> 8;         // High byte

    while(1);
    return 0;
}
```

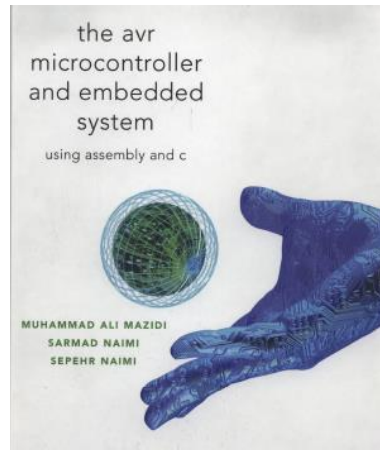
TIMER 1: IC

- Ejemplo: Medición de **Temperatura y Humedad relativa** con sensor digital

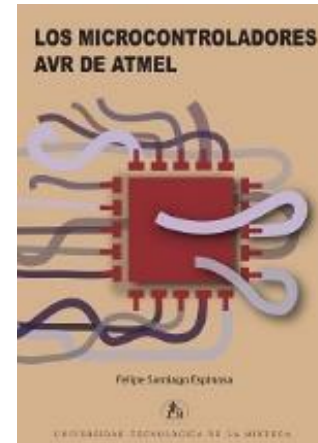


Bibliografía:

- *The AVR microcontroller & Embedded Systems*. Mazidi,Naimis, CH15



- Libro Digital de Felipe Espinosa, CH4



- Hojas de datos ATMEGA328p, 2560